Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
МГТУ им. Н.Э. Баумана

Презентация к курсовой работе на тему

Разработка базы данных медицинской клиники

Студент: ИУ7-62Б Атаев Константин Александрович
Руководитель: Солодовников Владимир Игоревич
2026 г.

Цель и задачи

Цель работы — разработать базу данных для хранения информации о пациентах, врачах, расписании приема, записях, приемах, счетах и оплатах медицинской клиники.

Задачи:

- провести анализ предметной области медицинской клиники;
- сформализовать роли пользователей и основные операции системы;
- спроектировать сущности базы данных и связи между ними;
- реализовать ограничения целостности, функции и триггеры PostgreSQL;
- разработать backend, frontend и интерфейс доступа к базе данных;
- проверить корректность работы и исследовать производительность.

Анализ существующих решений

Критерий	Медицинские ИС	Универсальные CRM	Электронные таблицы
Хранение расписания врачей	Штатный календарь врача и кабинета.	Календарь и задачи без медицинских правил.	Таблица расписания, заполняемая вручную.
Запрет пересечения записей	Обычно реализован на уровне системы.	Требует настройки правил или доработки.	Контролируется пользователем вручную.
История цен услуг	Поддерживается через прайс-листы и периоды действия.	Может храниться в пользовательских полях.	Есть риск перезаписи старых значений.
Проведение приема	Есть карточка приема, диагноз и рекомендации.	Не является базовой сущностью CRM.	Обычно ведется на отдельных листах.
Поддержка счетов и оплат	Счета, строки счетов и статусы оплаты.	Частично через сделки, счета или интеграции.	Ручной расчет и ручное изменение статусов.
Ролевая модель	Пациент, врач, администратор и другие роли.	Роли интерфейса без явной модели прав БД.	Ограничение доступа только к файлу или листу.
Аудит и проверка данных	Журнал действий и отчеты.	Частично зависит от настроек CRM.	Проверка затруднена, ошибки легко внести вручную.

Разрабатываемая база данных должна хранить медицинские сущности и контролировать правила предметной области: расписание, цены, приемы, счета, оплату и права доступа.

Диаграмма сущность-связь базы данных в нотации Чена

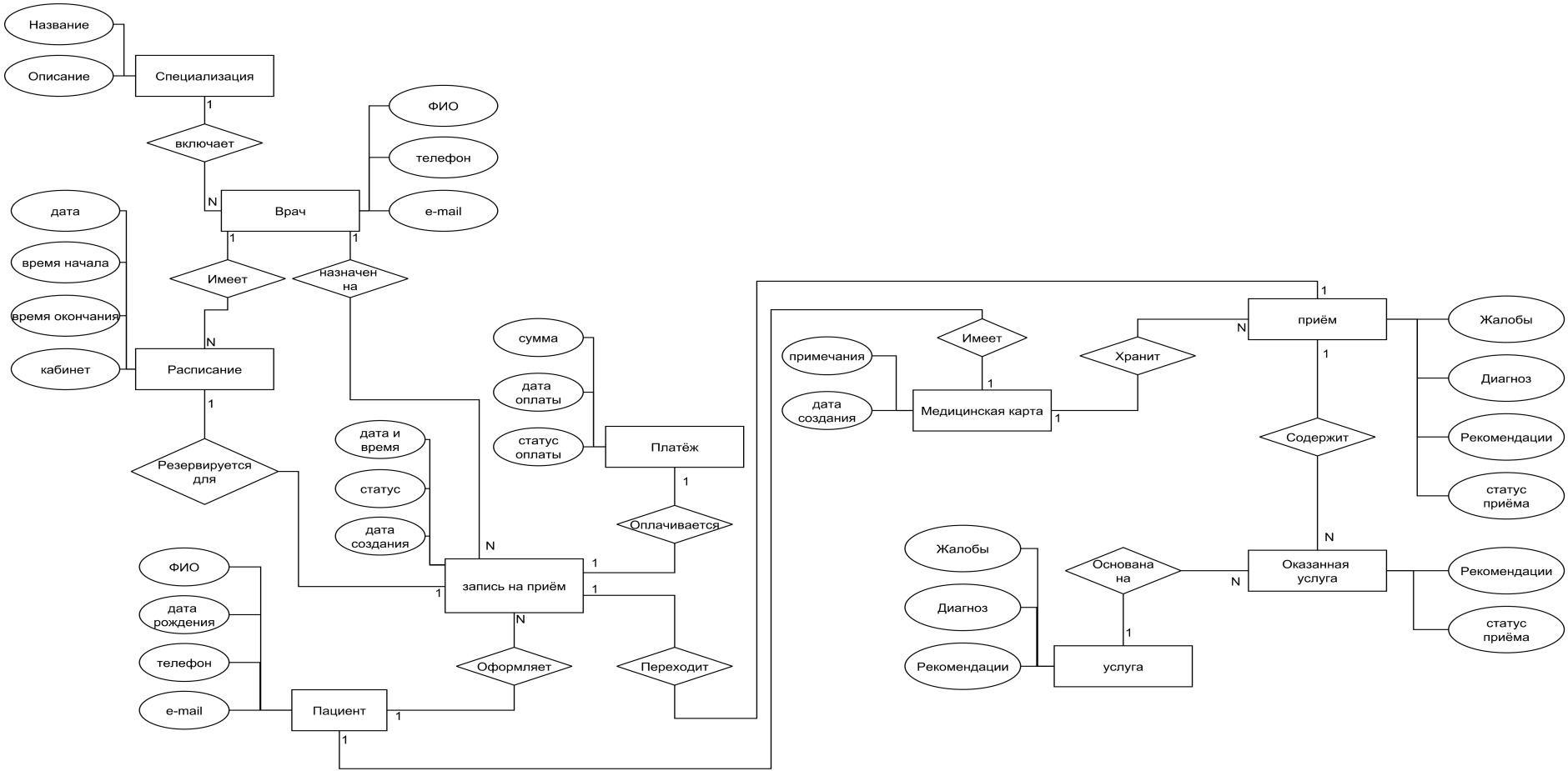


Схема проектируемой базы данных

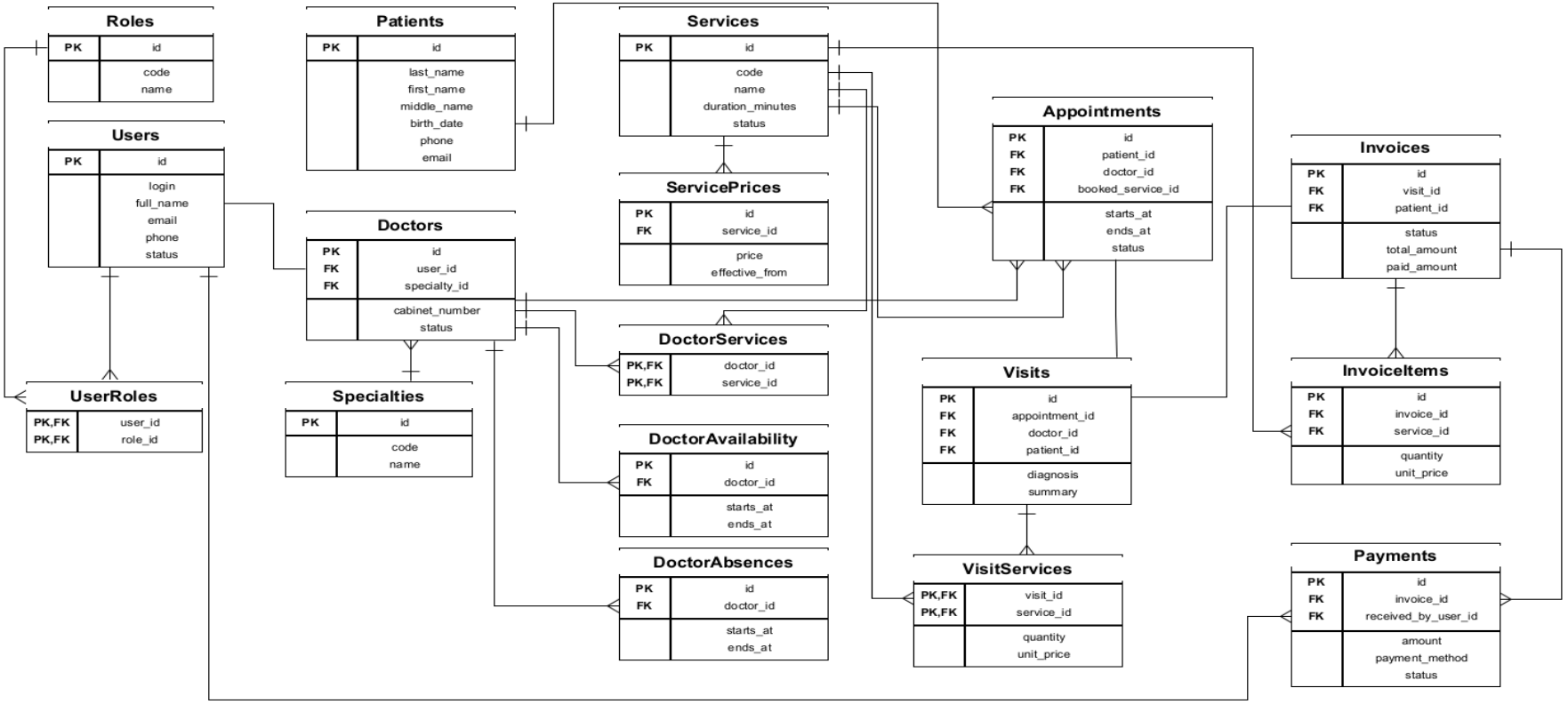


Схема связывает роли, справочники услуг, расписание, приемы, счета и оплаты.

Диаграмма прецедентов



Актеры расположены вне границ системы, прецеденты — внутри контура приложения.

Классификация моделей данных

Модель	Особенности	Применимость к задаче
Иерархическая	данные представлены деревом	неудобна для связей врачей, услуг и приемов
Сетевая	запись может иметь несколько связей	сложна в сопровождении
Реляционная	таблицы, ключи, ограничения, SQL	выбрана для реализации
Многомерная	ориентирована на OLAP-анализ	не подходит для основной OLTP-системы
Объектная	хранение объектов и методов	не дает преимуществ для данной задачи

Анализ модели данных и СУБД

Для реализации выбрана реляционная модель данных.

Причины выбора:

- поддержка нормализации и внешних ключей;
- наличие транзакций и ограничений целостности;
- возможность создания индексов, представлений, функций и триггеров;
- поддержка диапазонных типов PostgreSQL для работы с временными интервалами;
- удобство интеграции с backend-приложением через SQL-запросы.

Критерий	Иерарх.	Сетевая	Реляц.	Многомер.	ОО
Связи M:N	нет	да	да	нет	да
Декларативные запросы	нет	нет	да	частично	нет
Ограничения целостности	частично	частично	да	нет	частично
Транзакционная обработка	частично	частично	да	нет	да
Распространенность СУБД	низкая	низкая	высокая	средняя	низкая

Реализованные ограничения целостности

- первичные и внешние ключи связывают пользователей, врачей, пациентов, записи, приемы и счета;
- домены проверяют формат e-mail и номера телефона;
- CHECK-ограничения запрещают отрицательные суммы и некорректные интервалы времени;
- TSTZRANGE используется для интервалов цен, доступности врача и записей;
- EXCLUDE-ограничения запрещают пересечение активных записей одного врача;
- триггеры пересчитывают состояние счета после изменения строк счета или оплат.

Реализованные функции базы данных

Функция	Назначение
get_service_price	получение актуальной цены услуги на заданный момент времени
book_appointment	запись пациента на прием после проверки доступности врача
cancel_appointment	отмена записи с учетом статуса и роли пользователя
complete_visit	проведение приема, фиксация оказанных услуг и формирование счета
register_payment	регистрация оплаты по счету
refresh_invoice_state	пересчет суммы, оплаченной части и статуса счета

Схема алгоритма функции завершения приёма пациента (complete_visit) часть 1

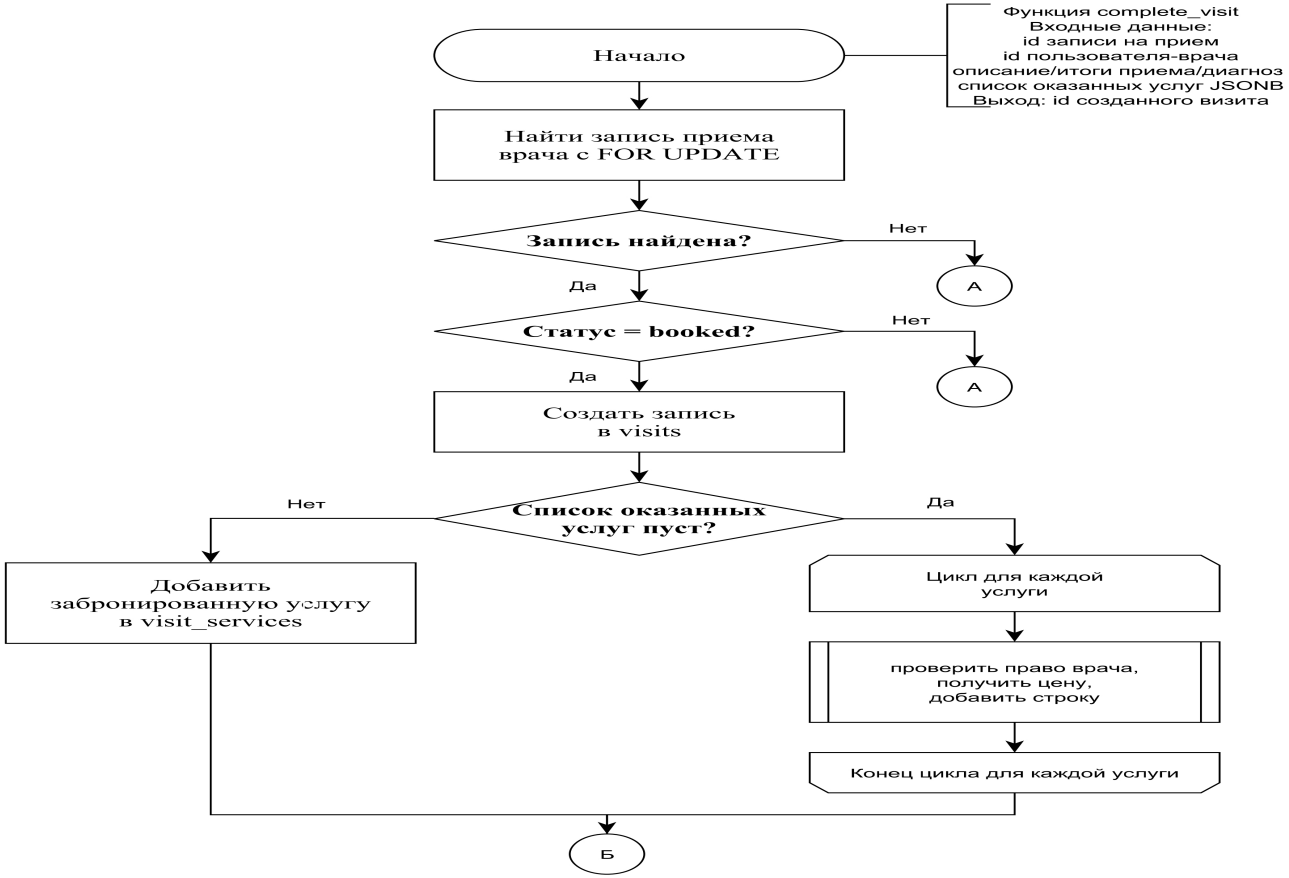


Схема алгоритма функции завершения приёма пациента (complete_visit) часть 2

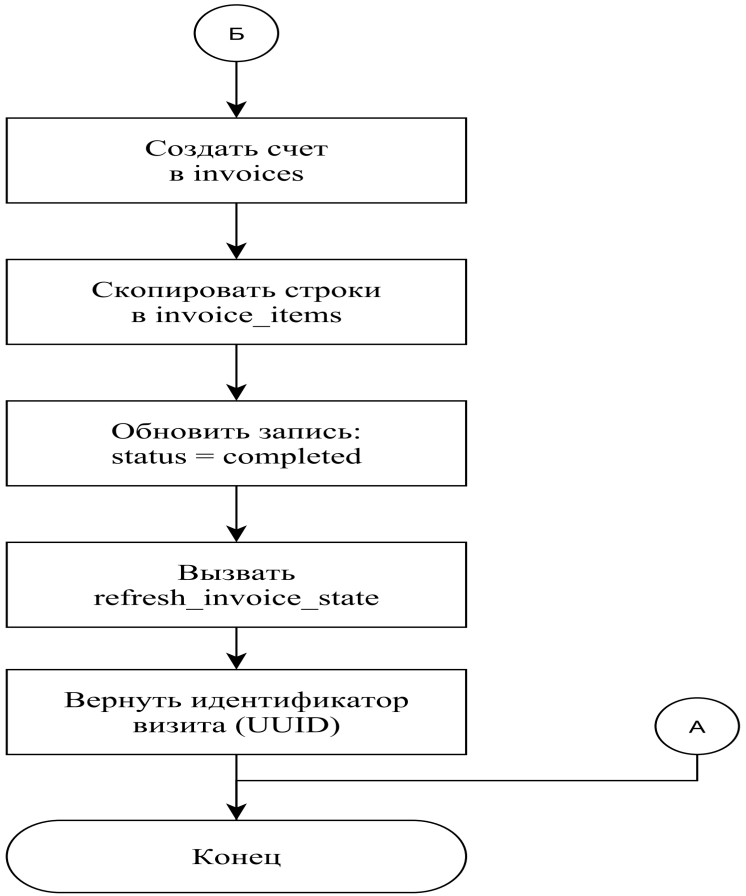
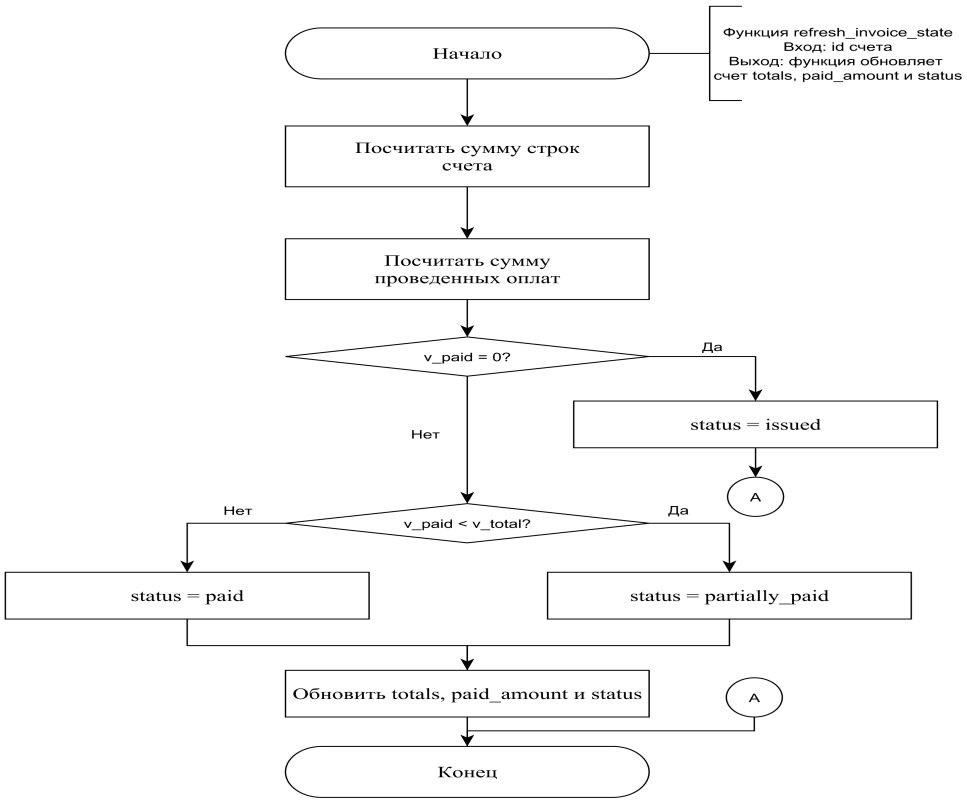
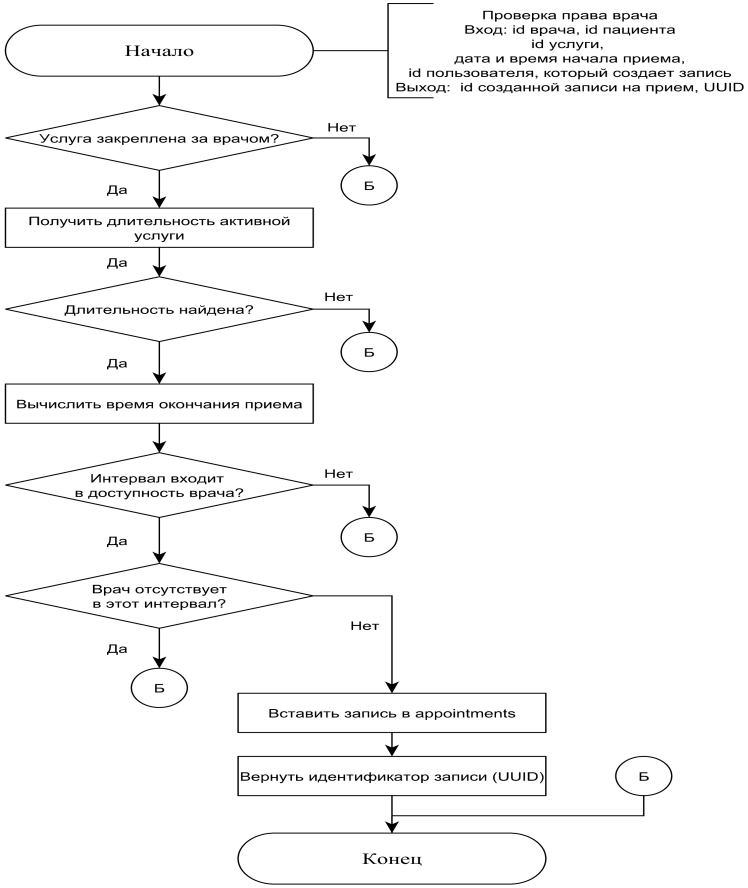




Схема алгоритма функции завершения приёма пациента (complete_visit) часть 3





Средства реализации

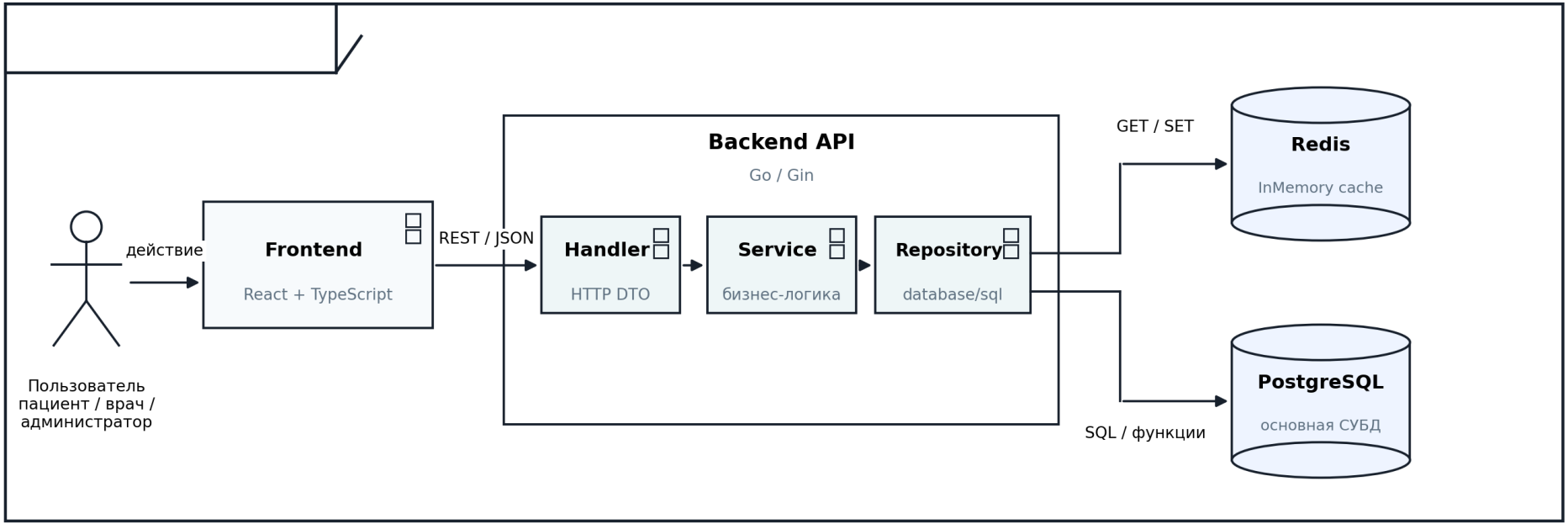
Стек реализации

Средство	Назначение
PostgreSQL	основная СУБД, хранение данных и функции
Redis	кэш расписания и прайс-листа
Go, Gin	backend и REST API
React, TS	frontend-интерфейс
Docker	контейнерный стенд проекта

Выбор основной БД

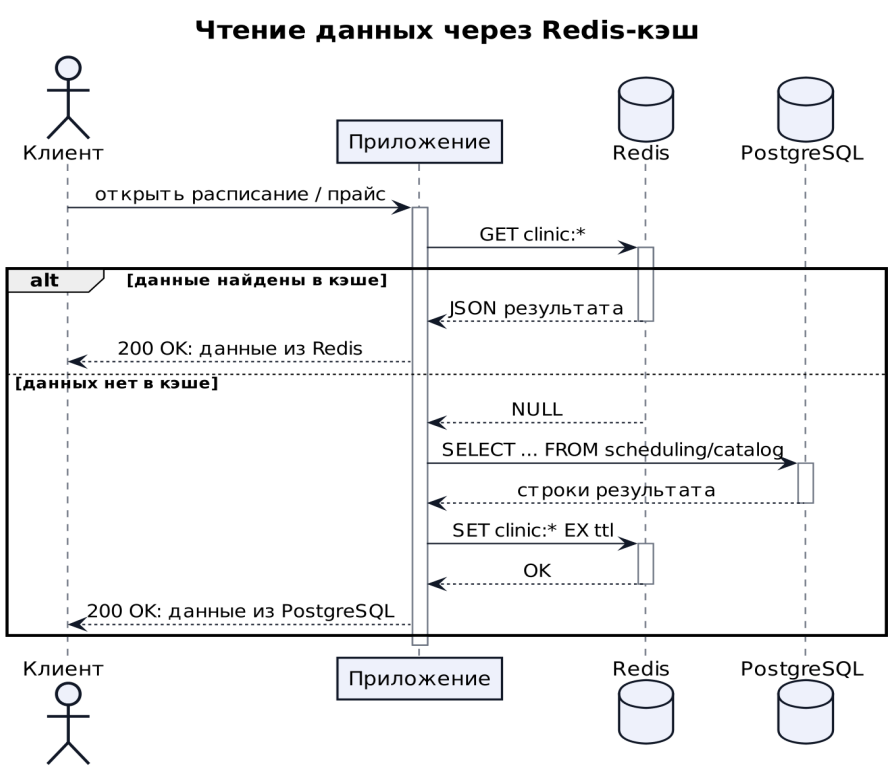
Критерий	PostgreSQL	Зачем
Реляционная модель	да	связи и нормализация
Транзакции, FK	да	согласованность
TSTZRANGE, EXCLUDE	да	запрет конфликтов
Функции, триггеры	да	логика в БД

Схема программного обеспечения



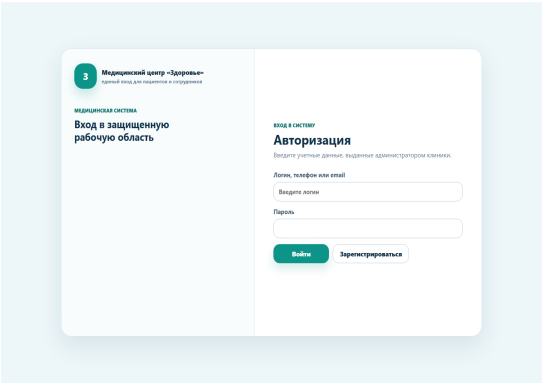
Frontend работает через REST API. Backend разделен на Handler, Service и Repository; основные данные хранятся в PostgreSQL, Redis используется для кэширования.

Диаграмма взаимодействия с реляционной СУБД и InMemory СУБД

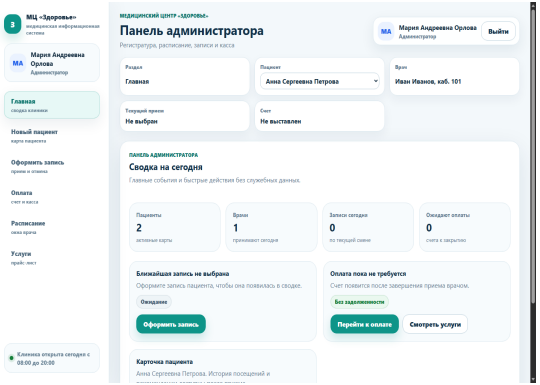




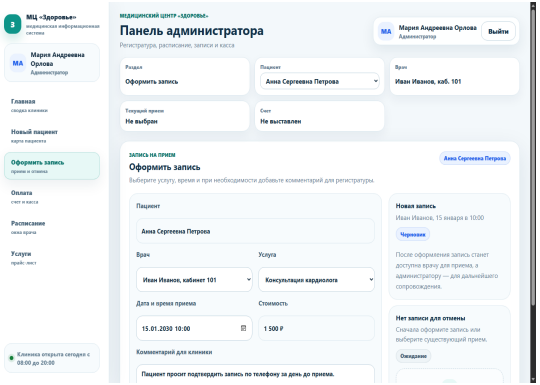
Демонстрация работы приложения



Вход в приложение



Рабочее место пользователя

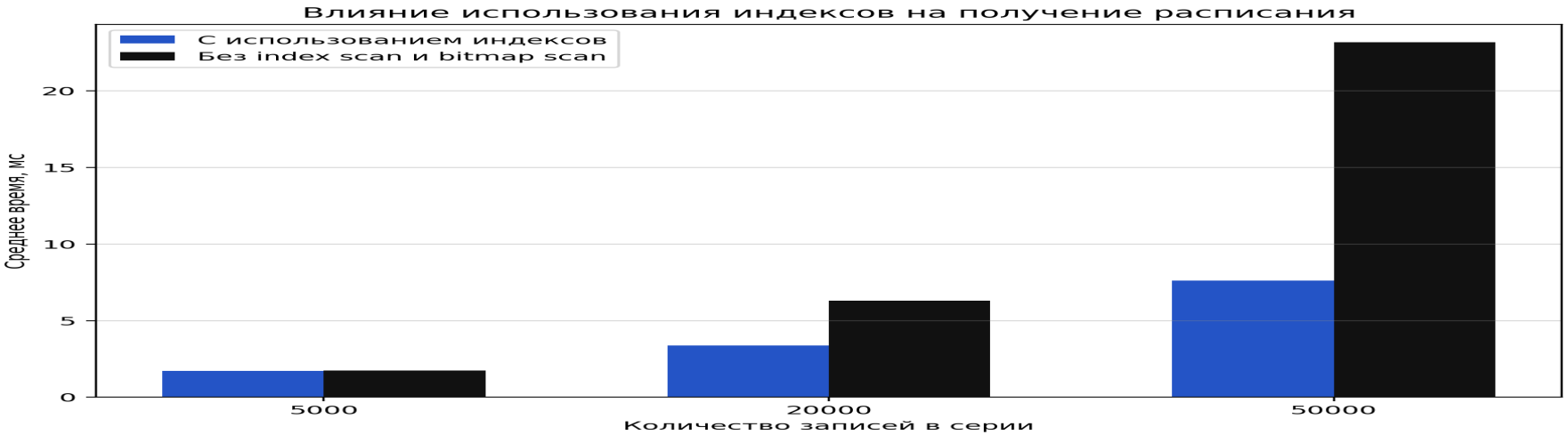


Оформление записи

Интерфейс адаптируется под роль пользователя: пациент работает с записью, врач проводит прием, администратор сопровождает расписание и оплату.

Исследование зависимости времени чтения расписания от использования индексов

Методика: EXPLAIN ANALYZE внутри PostgreSQL; 7 запусков на каждый объем.

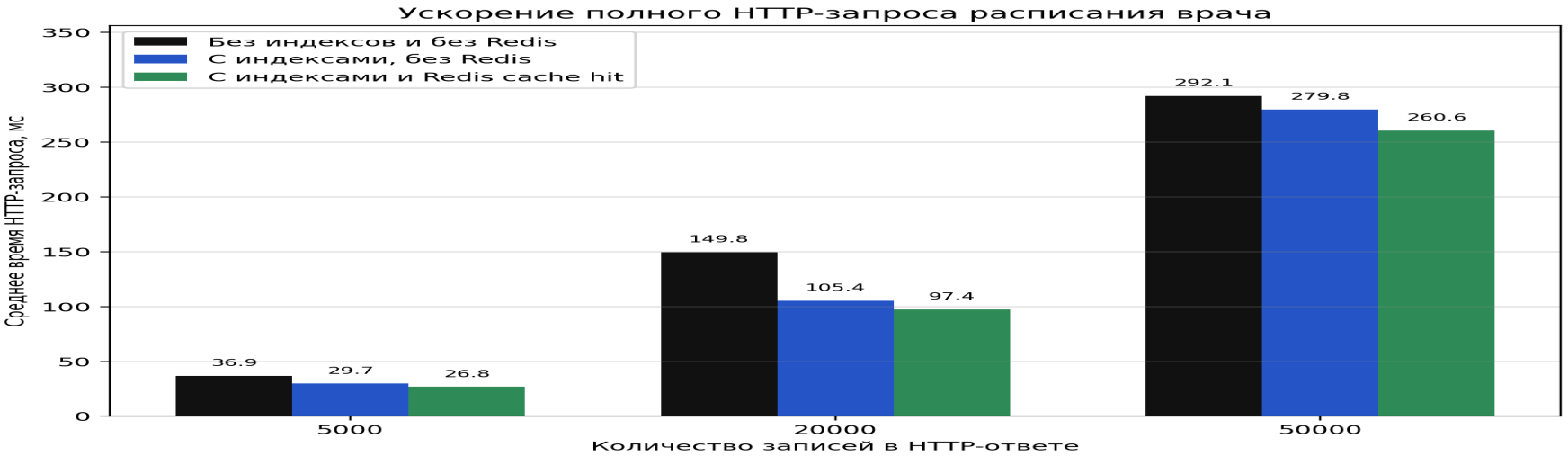


Объем	Без инд., мс	С инд., мс	Выигрыш, %
5 000	1,745	1,717	1,60
20 000	6,292	3,387	46,17
50 000	23,188	7,614	67,16

Индексы дают основной выигрыш на больших объемах: для 50 000 строк чтение расписания ускорилось примерно в 3 раза.

Исследование зависимости времени чтения расписания от использования индексов

Сравнение цепочки: без индексов -> индексы -> Redis cache hit.

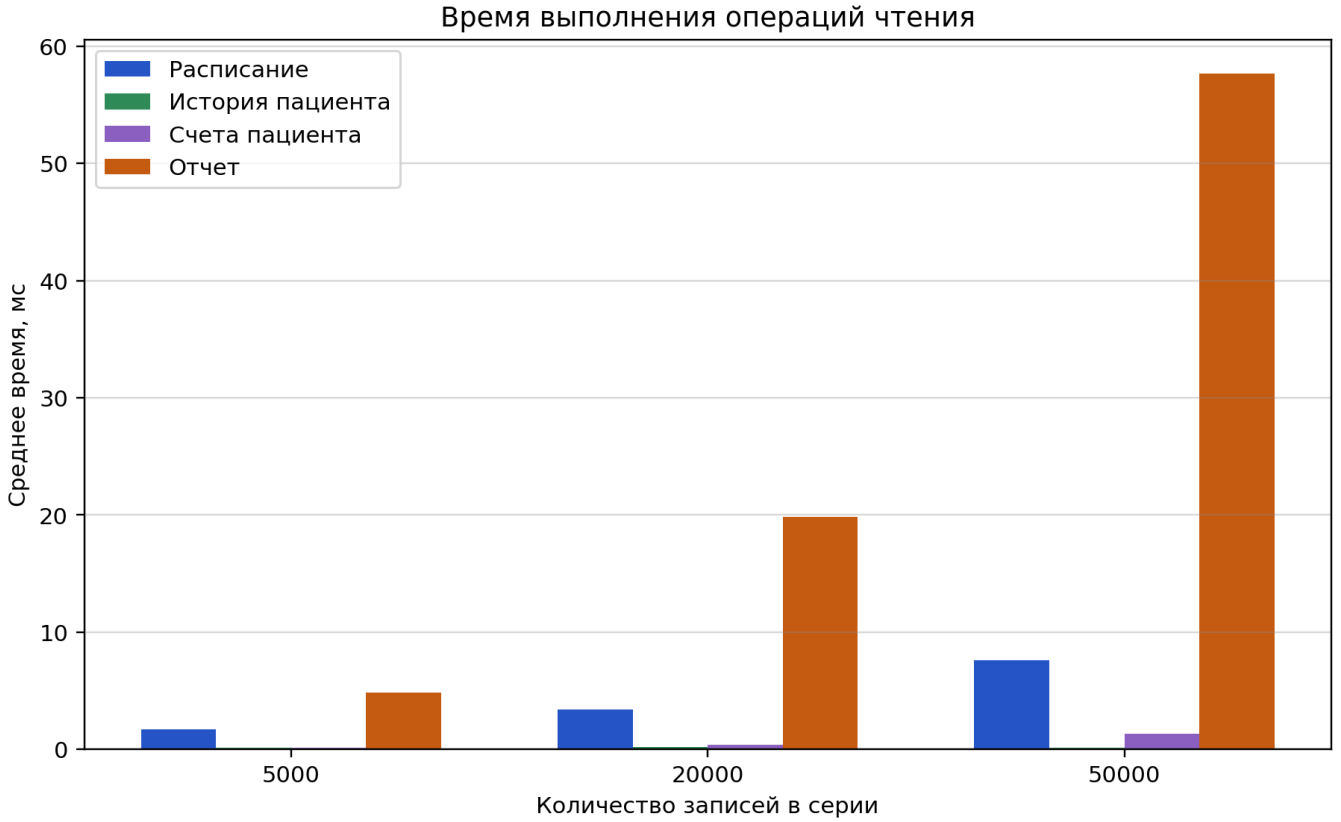


Объем	Без инд., мс	С инд., мс	Redis hit, мс
5 000	36,893	29,719	26,801
20 000	149,756	105,388	97,442
50 000	292,108	279,806	260,563

Замер показывает пользовательский путь целиком: HTTP, backend, PostgreSQL или Redis, сериализацию JSON и передачу ответа.

Исследование зависимости времени операций чтения от объема данных

Сравнение расписания, истории пациента, счетов и отчета при росте объема данных.



Самой тяжелой операцией оказался отчет по выручке: он соединяет несколько таблиц и выполняет агрегацию.

Заключение

Цель работы достигнута: разработана база данных медицинской клиники и приложение доступа к ней.

В ходе работы:

- проведен анализ предметной области и существующих решений;
- спроектированы сущности базы данных, роли и ограничения целостности;
- реализованы функции, триггеры и ролевая модель PostgreSQL;
- разработаны backend, frontend и кэширование часто запрашиваемых данных;
- проверена корректность работы и исследована производительность системы.